MODULE 24 · SECTION 3 - STORING THE DATA

Modelling the Warehouse

Star schema or one big table, nested and repeated fields, layers of modelling, slowly changing dimensions, append-only history, and keys BigQuery does not enforce.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 24 of 45

admin@sthithapragna.com

Storage is cheap. Shuffles are not. **Model for the questions people ask.**

Star schema or one big table?

STAR SCHEMA

Facts and dimensions

- Narrow facts, keys to dimensions
- Change a dimension in one place
- Familiar to analysts and BI tools
- Joins happen at query time

DENORMALISED

One wide or nested table

- Everything a query needs, together
- No joins, so no shuffles
- Values repeat on many rows
- A change can touch many rows

THE TIEBREAKER

BigQuery runs both well. Ask which query runs most often, and which change happens most often.

Keep children inside the parent

- A **STRUCT** groups related fields. An **ARRAY** holds the many side of a one-to-many.
- Children sit beside their parent, so there is **no join and no shuffle**.
- Best when the data is **hierarchical** and always read together.
- Avoid for **high-cardinality** children. Nesting stops at 15 levels.

> THE NUMBERS
>
> In Google's own example, nesting cut a query from about 25 seconds to 10, and read less data.

Bills with their lines inside

```
 1 CREATE TABLE curated.bills_nested (
 2   bill_id      STRING NOT NULL,
 3   account_id   STRING NOT NULL,
 4   period_start DATE,
 5   amount_gbp   NUMERIC,
 6   lines ARRAY<STRUCT<
 7     line_type  STRING,     -- energy, standing charge, VAT
 8     kwh        NUMERIC,
 9     amount_gbp NUMERIC>>)
10 PARTITION BY DATE_TRUNC(period_start, MONTH)
11 CLUSTER BY account_id;
```

6-9 Three to eight lines per bill, stored inside the bill row.

10 Monthly partitions: a few million bills a month is too little for daily.

FOR LARKSPUR

Two point two million accounts make a few million bills a month, so monthly partitions stay well sized.

Reading the lines, with no join

```sql
-- Standing charges per bill this year, read straight from the bill rows
SELECT b.bill_id, SUM(l.amount_gbp) AS standing_gbp
FROM curated.bills_nested AS b, UNNEST(b.lines) AS l
WHERE b.period_start >= '2026-01-01'
  AND l.line_type = 'standing charge'
GROUP BY b.bill_id;
```

3 UNNEST turns the array into rows, inside each bill. Nothing is shuffled.

REMEMBER

A query that ignores the lines never reads them. Columnar storage skips unused fields.

One warehouse, three layers

01

Raw

As the source sent it. Append-only, duplicates allowed.

02

Curated

Cleaned and deduplicated. Star or nested, by subject.

03

Marts

Wide tables for one audience, such as Westgrid.

04

Views

Stable names on top, while tables underneath change.

THE PATTERN

The degree of normalisation changes by layer. There is no single right answer for the whole warehouse.

Move first, then improve

- Google advises moving the schema **unchanged** in the first iteration.
- Upstream pipelines and existing reports keep working, and tools can automate it.
- Then evolve in stages: nested fields, **partitioning** and **clustering**.
- **Facade views** keep old names working while the tables change.

> THE TRAP
>
> Keeping the old model forever keeps its technical debt. Lift first, but plan the next step.

When a dimension changes

APPROACH	WHAT HAPPENS	HISTORY KEPT
Type 1	Overwrite the old value	None
Type 2	Add a row with valid-from and valid-to	Full, row by row
Type 3	Keep a previous-value column	One step back
Daily snapshot	Copy the whole dimension each day	By date

THE SIGNAL

'What was it at the time?' needs Type 2 or snapshots. 'Fix a typo' is Type 1.

admin@sthithapragna.com

Which tariff was this bill on?

```sql
-- Join each bill to the tariff that was live when it started
SELECT b.bill_id, b.period_start, h.tariff_id
FROM curated.bills AS b
JOIN curated.account_tariff_history AS h
  ON  h.account_id = b.account_id
  AND b.period_start >= h.valid_from
  AND b.period_start <  h.valid_to;   -- open rows use DATE '9999-12-31'
```

6-7 A point-in-time join: the version whose dates cover the bill.

REMEMBER

Use a far-future date for the current row, not NULL. Then one condition covers every row.

Append changes, then resolve them

- BigQuery is happiest **appending**. Many small updates are slow and costly.
- Datastream's **append-only** mode keeps every change as a new row.
- A view picks the **latest version** of each key, and drops deletes.
- Merge mode keeps a current copy instead, and needs a **source primary key**.

> THE PAYOFF
>
> Appending keeps the full history for free, and leaves old partitions untouched.

The current state of every account

```
1 CREATE OR REPLACE VIEW curated.accounts_current AS
2 SELECT * EXCEPT (rn) FROM (
3   SELECT *, ROW_NUMBER() OVER (PARTITION BY account_id
4               ORDER BY datastream_metadata.source_timestamp DESC,
5                        datastream_metadata.change_sequence_number DESC) AS rn
6   FROM raw.accounts_changes
7   WHERE datastream_metadata.change_type != 'UPDATE-DELETE')
8 WHERE rn = 1 AND datastream_metadata.change_type != 'DELETE';
```

3-5 Number each account's changes, newest first.

7-8 Skip before-images, keep the newest, and drop deleted accounts.

HONEST LIMIT

Check how your source's sequence numbers sort before trusting them as a tiebreaker.

Keys that BigQuery does not enforce

- Primary and foreign keys are declared **NOT ENFORCED**. Nothing is checked.
- They feed the optimiser: **join elimination** and better join order.
- If the data breaks them, queries can return **wrong results**.
- Deduplicate in the pipeline. The key describes the data, it does not police it.

> THE TRAP
>
> Declaring a primary key does not stop duplicates. The exam likes to test this.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Children **always read with** the parent	Nested and repeated fields
Joins **shuffle** and run slowly	Denormalise, or nest the child
What was it **at the time**?	Type 2, valid-from and valid-to
Keep **every change** from the source	Append-only, then a latest view
Migrate without breaking reports	Move as-is, then evolve behind views
Expect the key to **reject duplicates**	Not enforced: dedupe in the pipeline

THE PATTERN

Shape follows the question: what is read together, and what must be remembered.

> The most common billing query joins curated.bills to curated.bill_lines, billions of rows, and spends most of its time shuffling.
>
> Lines are always read together with their bill, and each bill has three to eight lines.

What redesign fits best?

A Cluster both tables on bill_id, and keep the join as it is

B Nest the lines inside each bill, as a repeated STRUCT

C Normalise further, moving line types into a table of their own

D Move the bill lines into Bigtable, with bill_id as the row key

◆ PAUSE AND DECIDE

What redesign fits best?

A Cluster and keep the join

Helps a little, but the join still shuffles billions of rows.

B Nest the lines

No join, no shuffle: lines sit inside their bill.

C Normalise further

Adds another join to a query already slowed by joins.

D Bigtable

An operational store, not for analytical SQL.

THE SIGNAL

Children always read with the parent, few per parent: nest them.

> The complaints team must show which tariff an account was on when each bill was issued, going back years.
>
> Accounts change tariff about once a year. Today, the accounts table simply overwrites tariff_id.

What should the team change?

A. A Type 2 tariff history, with valid-from and valid-to dates

B. Keep overwriting, and use time travel to read older tariff values

C. Add a previous_tariff_id column to the accounts table

D. Copy tariff_id onto every half-hourly reading as it is loaded

◆ PAUSE AND DECIDE

What should the team change?

A Type 2 history

Every version kept, joined by date to each bill.

B Time travel

Seven days at most, not years.

C Previous-value column

Type 3 looks back one change only.

D Copy onto readings

Wrong table, billions of copies, and no history for the past.

THE SIGNAL

What was it at the time, years back: a Type 2 dimension.

> An engineer declared PRIMARY KEY (account_id) NOT ENFORCED on curated.accounts, expecting it to block duplicates from the CDC feed.
>
> A week later, duplicate accounts appear, and some join results are wrong.

What should the team do?

A. Recreate the key as ENFORCED, so that BigQuery rejects duplicates

B. Deduplicate upstream, as the key only guides the optimiser

C. Drop the key, because primary keys slow down BigQuery joins

D. Cluster the table on account_id, so duplicate rows merge on write

◆ PAUSE AND DECIDE

What should the team do?

A Enforced key

BigQuery does not offer enforced primary keys.

B Deduplicate upstream

The key is a promise to the optimiser. The pipeline must keep it.

C Drop the key

Keys speed up joins when the data honours them.

D Cluster on the key

Clustering sorts rows. It never merges them.

THE SIGNAL

Keys are not enforced: deduplicate in the pipeline, or results can be wrong.

> Larkspur must move 50 Oracle billing tables to BigQuery before the data-centre lease ends.
>
> Existing reports must keep working, and the team wants a nested model in the long run.

What approach should the team take?

A. Move the schema as it is, then evolve it behind views

B. Redesign every table as nested before any data is moved

C. Flatten all fifty tables into one wide table during the move

D. Keep Oracle for reports, and copy only aggregates to BigQuery

◆ PAUSE AND DECIDE

What approach should the team take?

A Move, then evolve

Reports keep working. Views hide each later change.

B Redesign first

Breaks reports and pipelines, and risks the deadline.

C One wide table

Huge repetition, and every report must be rewritten.

D Keep Oracle

The data centre is closing, so Oracle cannot stay.

THE SIGNAL

A deadline plus reports that must keep working: move as-is, then evolve behind views.

Module 24 in five lines

- Storage is cheap, shuffles are not. Model for the **commonest query**.
- **Nest** children that are always read with their parent, and few per parent.
- **Type 2** rows with valid-from and valid-to answer 'what was it then?'.
- **Append** every change, and resolve the current state with a view.
- Keys are **not enforced**. Move schemas as-is first, then evolve.

COMING UP

Next: module 25, lakes and lakehouses.